Тема 03. Типы и модели данных

Блок 1 · Предыдущий: 02 · Следующий: 04. ООП и паттерны

> **Время:** 12–18 часов на три шага — 03.1 (3–5 ч), 03.2 (4–6 ч), 03.3 (5–7 ч).
> Обоснование — TIME.md. Незнакомые слова — словарь.
> Шаг 03.1 специально сделан лёгким: он про порядок в уже написанном коде, без новых понятий.

Зачем эта тема

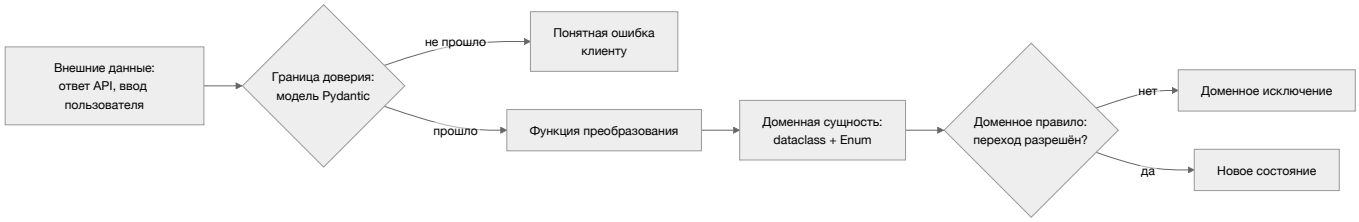
Типы — это проверяемая документация: они ловят ошибки до запуска и делают код читаемым. Здесь же появляются первые сущности вашей предметной области, вокруг которых вырастет проект.

Тема идёт в три приёма: сначала навести порядок в готовом коде, потом описать свои сущности, и только потом включить требовательную проверку типов. Так ошибки языка и ошибки библиотеки не приходится разбирать одновременно.

Теория

- **Аннотации типов** — пометки о типах аргументов и возвращаемых значений. Интерпретатор их не проверяет, проверяет отдельный инструмент.
- **mypy** — программа, которая читает аннотации и находит несоответствия до запуска кода.
- **Строгий режим** (`strict`) — набор самых требовательных проверок mypy. Он не запрещает автоматически все источники `Any`; публичные сигнатуры проверяйте отдельно.
- **dataclasses** — компактный способ описать объект с полями без валидации. Подходит для внутренних доменных сущностей.
- **Pydantic v2** — валидация и разбор данных на основе аннотаций. Нужен на границе приложения: там, где данные приходят извне и им нельзя доверять.
- **Граница доверия** — внешнюю структуру проверяют при входе. Доменные правила проверяют при каждой операции: корректный формат статуса ещё не означает допустимый переход.

Где какая проверка происходит:



Pydantic отвечает на вопрос «правильной ли формы данные», домен — на вопрос «допустимо ли это действие». Первое проверяется один раз на входе, второе — при каждой операции.

Синтаксис аннотаций

```
from collections.abc import Callable
from typing import Literal, TypedDict

def find(titles: list[str], query: str) -> str | None: ...   # строка или None
counts: dict[str, int] = {}
point: tuple[int, str] = (1, "a")      # ровно два элемента
ids: tuple[int, ...] = (1, 2, 3)       # любое число элементов одного типа
rule: Callable[[dict], bool]           # функция: принимает dict, возвращает bool
Order = Literal["asc", "desc"]         # только эти два значения

class RawVacancy(TypedDict):           # словарь с известными ключами, например JSON
    title: str
    salary: int | None

def first[T](items: list[T]) -> T:     # обобщённая функция: тип результата
    return items[0]                    # совпадает с типом элементов
```

`Any` отключает проверку: с таким значением mypy разрешает всё. `object` — наоборот, «что угодно, но сначала проверь тип»: без `isinstance` с ним ничего нельзя сделать. Поэтому для неизвестных данных честнее `object`, а `Any` в публичной сигнатуре распространяет непроверяемость на весь вызывающий код.

Как mypy рассуждает и как читать его ошибки

```
app/select.py:12: error: Argument 1 to "find" has incompatible type "int";
expected "str"  [arg-type]
```

Файл и строка, суть несоответствия, в скобках — код ошибки. mypy **сужает** типы по условиям: после `if title is None: return` он знает, что `title` — строка. `reveal_type(выражение)` печатает выведенный тип. Подавляют ошибку точечно и с причиной: `# type: ignore[arg-type]`.

Строгий режим включает набор флагов: запрет функций без аннотаций, запрет неявного `Any` в обобщённых типах, предупреждение о возврате `Any` и другие. Флаг `strict` задаётся только для всего запуска, поэтому строгость для одного модуля включают либо отдельной командой `mypy --strict src/job_tracker/domain`, либо перечислением отдельных строгих флагов в секции `[[tool.mypy.overrides]]` для этого модуля.

dataclass и Enum

```
from dataclasses import dataclass, field
from enum import StrEnum

class Status(StrEnum):
    SAVED = "saved"
    APPLIED = "applied"

@dataclass(frozen=True, slots=True)
class Salary:                          # значение: сравнивается по содержимому
    amount: int
    currency: str = "RUB"

@dataclass
class Application:                     # сущность: состояние меняется методами
    vacancy_id: int
    status: Status = Status.SAVED
    notes: list[str] = field(default_factory=list)   # не notes = []
```

`@dataclass` генерирует `__init__`, `__repr__` и `__eq__` по списку полей. `frozen=True` запрещает присваивание атрибутов после создания и делает объект хешируемым. `slots=True` убирает словарь атрибутов: объект занимает меньше памяти, а присваивание несуществующему полю из-за опечатки сразу падает. Проверки при создании пишут в методе `__post_init__`. Изменяемое значение по умолчанию задают только через `default_factory`: иначе один список делили бы все экземпляры, ровно как в ловушке из подготовки.

`Enum` — закрытый набор значений. Опечатка `Status.APLIED` падает сразу, mypy проверяет, что передан именно статус, а `Status("unknown")` выбрасывает `ValueError` на границе. `StrEnum` вдобавок равен своей строке: `Status.SAVED == "saved"`, что удобно для JSON и БД.

Отсутствующее поле, None и значение по умолчанию

```
class VacancyIn(BaseModel):
    title: str                 # обязательное: без ключа ошибка
    salary: int | None         # ключ обязателен, но значение может быть null
    city: str | None = None    # ключ можно не передавать: тогда None
    remote: bool = False       # ключ можно не передавать: тогда False
```

`X | None` описывает допустимые значения, а обязательность задаёт наличие значения по умолчанию. Различие становится важным при частичном обновлении: «поле не передали» значит «не трогать», а «передали null» — «очистить». Какие поля пришли на самом деле, показывает `model.model_fields_set`.

Pydantic на границе доверия

```
from pydantic import BaseModel, ConfigDict, Field, HttpUrl, field_validator

class VacancyIn(BaseModel):
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)

    external_id: str = Field(pattern=r"^[A-Z]{2}-\d{6}$")
    title: str = Field(min_length=1, max_length=200)
    salary: int | None = Field(default=None, ge=0)
    url: HttpUrl

    @field_validator("title")
    @classmethod
    def not_only_digits(cls, value: str) -> str:
        if value.isdigit():
            raise ValueError("название не может состоять из одних цифр")
        return value

vacancy = VacancyIn.model_validate(raw)   # при ошибке — ValidationError
```

`ValidationError` собирает все ошибки сразу, а не первую: `err.errors()` возвращает список с путём к полю, типом ошибки и сообщением. `extra="forbid"` отвергает неизвестные ключи, и опечатка в имени поля не проходит молча. По умолчанию Pydantic приводит совместимые значения: строка `"42"` станет числом 42. Режим `strict=True` такое приведение запрещает. После проверки модель превращается в доменную сущность обычной функцией: входная модель знает про формат внешних данных, доменная — только про смысл.

Регулярные выражения: почему fullmatch

`re.match` проверяет только начало строки, `re.search` ищет совпадение где угодно, `re.fullmatch` требует, чтобы шаблону соответствовала вся строка. `re.match(r"\d{6}", value)` пропустит `"123456abc"`, а `$` в конце шаблона разрешает перевод строки перед концом. Для проверки формата

идентификатора берут `re.fullmatch(r"[A-Z]{2}-\d{6}", value)` и пишут шаблон сырой строкой `r"..."`, чтобы обратная косая черта не обрабатывалась дважды.

Материалы

- typing: документация — как справочник, не подряд.
- mypy cheat sheet — сжатая выжимка синтаксиса.
- dataclasses — параметры `frozen`, `slots`, `field`.
- Pydantic: Models — модели, валидаторы, режимы.

Задание

Шаг 03.1. Аннотации и порядок в готовом коде (3–5 ч)

Новых библиотек здесь нет. Цель — привести в порядок код подготовки и увидеть, что типы находят настоящие ошибки. Если тема типов пока пугает, задержитесь именно на этом шаге: он самый безопасный для экспериментов.

1. Проставьте аннотации у функций, написанных в подготовке: что принимает, что возвращает. Достаточно простых типов: `str`, `int`, `list[str]`, `dict[str, str]`, `X | None`.
2. Установите mypy в dev-группу и запустите его **без строгого режима**. Исправьте найденное. На каждую ошибку сформулируйте своими словами, что именно не сходится.
3. Разбейте одну самую длинную функцию на две-три по смыслу, не меняя поведения. Проверьте, что демонстрационный запуск даёт тот же результат, что до правки.
4. Дайте понятные имена переменным и функциям средствами рефакторинга IDE (приём из шага 00.3).
5. Запишите одну ошибку, которую нашёл mypy и которую вы бы не заметили сами. Если такой не оказалось — так и запишите, это тоже результат.

Шаг 03.2. Доменные сущности и перечисления (4–6 ч)

1. Начните с 2–3 сущностей из паспорта: поля, типы, обязательность. Остальные сущности и связи пока набросайте схемой; они появятся вместе с использующей их функцией.
2. Реализуйте их как `dataclass`. Неизменяемость выбирайте для значений, которым не нужен жизненный цикл; изменяемая сущность допустима, если её состояние меняется через проверяемый метод.
3. Добавьте перечисления (`Enum`) для статусов и типов вместо строковых констант.
4. Объясните разницу между отсутствующим полем, `None` и значением по умолчанию — на примере своих же сущностей.
5. Убедитесь, что доменные сущности не содержат работы с сетью, файлами и базой данных.

Шаг 03.3. Входные схемы и проверка типов (5–7 ч)

1. Входные данные (то, что придёт из внешнего источника или от пользователя) опишите моделями Pydantic с валидацией: диапазоны, форматы, обязательные поля.
2. Напишите функцию преобразования внешней модели в доменную сущность.

3. Включите строгий режим mypy **только для модуля с доменными сущностями и моделями** и исправьте ошибки по одной. Для каждой найдите понятную причину. Остальной код пока проверяется обычным режимом: распространение строгости на весь пакет — желательный критерий.
4. Добавьте mypy в pre-commit и существующий CI. Хук должен видеть зависимости проекта: можно использовать локальный хук с `language: system`, `entry: uv run mypy src` и `pass_filenames: false`, чтобы pre-commit не добавлял к проверке всего пакета отдельные файлы повторно.
5. Проверьте формат одного доменного идентификатора через `re.fullmatch` и добавьте пример неверного значения. Для URL и email используйте готовые валидаторы, а не большую самописную regex.
6. На маленьких примерах разберите `Callable`, `Literal`, `TypedDict` и функцию, работающую с разными типами. В проекте применяйте их только при наличии задачи; `Protocol` будет на шаге 04.1.

Обязательные критерии

- ▢ mypy проходит без ошибок: строгий режим — для модуля сущностей и моделей, обычный — для остального кода. Массовых игнорирований нет.
- ▢ Невалидные входные данные отвергаются с понятным сообщением об ошибке.
- ▢ Статусы и типы выражены через `Enum`, а не строками.
- ▢ Доменные сущности не содержат логики работы с сетью, файлами или БД.
- ▢ Есть функция или метод преобразования внешнего представления в доменное.
- ▢ mypy проверяет один и тот же код локально и в CI; показан пример пойманной ошибки.
- ▢ Формат одного доменного идентификатора проверяется, неверное значение отклоняется.

Желательные критерии

- ▢ Строгий режим mypy распространён на весь пакет `src`.
- ▢ `Any` в публичных сигнатурах отсутствует либо каждое использование обосновано комментарием.
- ▢ Объяснена разница между отсутствующим полем, `None` и значением по умолчанию.
- ▢ Разобраны `Callable`, `Literal`, `TypedDict` и функция, работающая с разными типами.
- ▢ Записана ошибка, найденная mypy на шаге 03.1.

На что обратить внимание

- Изменяемый объект по умолчанию (`list`, `dict`) в аргументе функции — классическая ловушка.
- `X | None` (то же, что `Optional[X]`) означает «значение либо есть, либо `None`», а не «аргумент можно не передавать».
- Не тащите Pydantic в глубину домена: валидация нужна на входе, а не в каждом объекте.
- Строгий режим сразу на всём проекте выдаёт десятки ошибок и демотивирует. Включайте его по одному модулю: так каждая ошибка разбирается отдельно и запоминается.

Вопросы для собеседования

Проверьте себя по темам этого шага. Ответов в программе нет намеренно: формулировка своими словами и есть проверка. Где можно — отвечайте на примере своего кода.

1. Проверяет ли Python аннотации типов во время выполнения? Кто их проверяет?
2. Зачем нужны аннотации, если код работает и без них?
3. Что означает `X | None`? Значит ли это, что аргумент можно не передавать?
4. Что такое строгий режим mypy и почему его не включают сразу на весь проект?
5. Чем `dataclass` отличается от обычного класса и от модели Pydantic?
6. Что дают параметры `frozen` и `slots` в dataclass?
7. Где в приложении нужен Pydantic, а где он избыточен?
8. Что такое граница доверия и какие проверки происходят по каждую её сторону?
9. Почему статус выражают через `Enum`, а не строкой?
10. Зачем нужна отдельная функция преобразования внешней модели в доменную сущность?
11. Объясните разницу между отсутствующим полем, значением `None` и значением по умолчанию.
12. Для чего нужны `Literal` и `TypedDict`? Приведите по примеру.
13. Чем плох `Any` в публичной сигнатуре и что делать, если подходящего типа не видно?
14. Почему доменная сущность не должна знать о сети, файлах и базе данных?

Полный набор по блоку — вопросы по блоку 1.